Tutorial Sheet 3

Files

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
02.06.2024

Introduction

Question 1

Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one?

Question 2

If `/home/root/` is the current working directory, what is the absolute path name for the file whose relative path name is `../redha/BuS/sched.txt`?

Question 3

Contiguous allocation of files leads to disk fragmentation, because some space in the last disk block will be wasted in files whose length is not an integral number of blocks. Describe the internal and external fragmentation during the allocation and deletion of block.s

Question 4

A UNIX file system has 4 KiB blocks and 4-bytes disk addresses. What is the maximum file size in bytes if the inode contains 10 direct entries, and one single, double and triple indirect entry each?

Directory entries

`/bin`

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

`/usr/bin`

inode	entry
3855844	curl
3627359	ip
1698762	pkill
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

`/usr`

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

Question 5

Can you spot the hard links in these directory entries? What would the file `/bin/sh` contain if it was a symbolic link to `bash`?

Question 6

What would `/usr/lib64` contain if it was a symbolic link to the directory `/usr/lib`? Is this possible with a hard link?

Question 7

Name one advantage of symbolic link over hard links and one advantage of hard links over symbolic links.

Question 8

How many disk operations are needed to fetch the inode for a file with the path name `/usr/ast/courses/os/handout.t`? Assume that the inode for the root directory is in memory, but nothing else along the path is in memory. Also assume that all directories fit in one disk block.

How can we reduce that number of disk operations?

Basic file management

In this exercise, you will learn how to:

1. read the manual pages for system calls
2. access files at low level

1. Read the manual

Read and understand the `open` syscall on the manual, what are the arguments, what is the return value?

> **Tip**
>
> On the Linux terminal, type `man 2 open`. If you don't have access to a Linux terminal, all man pages can be found on the Internet.

How will you call this function to create a new file? What does the flag `O_TRUNC` is doing? How to combine multiple flags?

2. Open and modify a file

Write a C code that opens a file, that modifies all the occurrences of the char `'a'` to `'b'`, starting at the offset 592, until the end of file.

Understanding relations between file descriptor, file table and inodes

In this task, we'll be analyzing the state of file descriptors, file structures, and inodes in a parent and child process at specific points in the program execution.

```
1  int main()
2  {
3      char buffer[16];
4
5      int fd0 = open("file1.txt", O_RDONLY);
6      int fd1 = open("file2.txt", O_RDWR);
7      int fd2 = open("file3.txt", O_RDONLY, O_TRUNC);
8      int fd3 = dup(fd2);
9
10     lseek(fd0, 19, SEEK_SET);
11     lseek(fd1, 122, SEEK_SET);
12     lseek(fd2, 5, SEEK_SET);
13     read(fd3, buffer, 16);
14
15     if (fork() == 0) {
16         close(fd0);
17         close(fd2);
18
19         int fd4 = open("file5.txt", O_RDONLY);
20
21         lseek(fd4, 12, SEEK_SET);
22         write(fd1, BUFFER, 16);
23         lseek(fd3, -21, SEEK_CUR);
24
25         close(fd1);
26         close(fd3);
27         close(fd4);
28         return;
29     }
30
31     close(fd0);
32     close(fd1);
33     close(fd2);
34     close(fd3);
35 }
```

> **Tip**
>
> Check the man page for `dup` to understand line 8.

Please note that `file1.txt` and `file2.txt` are hard links. Suppose that no file descriptor is in use at the start of the program, except for the standard input, standard output and standard error.

Task

At the specified lines (14 and 24), draw the state of:

- The file descriptor table for each process.
- The associated file structures (including inode, mode, and cursor).
- The corresponding inode.

Questions

1. If the file permission of `file1.txt` was `0444`, what would be the result of executing the previous code?

Coding exercises

Dup 2 manipulation

> **Tip**
>
> In this task, you'll need `dup2` to do redirection. Check the man page, how is it different from `dup`?

Write a C function that executes a `command` with the arguments `args`. The function should call `fork()`, then executes the program with `execv()` and redirect the output of the command to the file `output_file`.

```
1  int stdout_to_file(char *command, char *args[], char *output_file)
2  {
3      ...
4  }
```

Implement the UNIX cat command

Implement the following C function that concatenate all the files passed in argument. It should write the content of the files to the standard output.

```
1  int cat(char *filenames[], int nr_files)
2  {
3      ...
4  }
```